

Islamic University of Technology

Lab 5: Dynamic Programming Engineering

CSE 4810: Algorithm Engineering
ASH, ABT(PT)
July 9, 2026

General Instruction

This lab consists of **One independent tasks**. For the task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file.
- Step 2.** Compile and run the program, feeding it the exact stress-test inputs specified.
- Step 3.** Produce the requested live artifact — a runtime plot, a memory log, or a comparison count — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen;

Files Provided

`Knapsack.cpp` 2D / 1D space-optimized 0-1 Knapsack skeleton
`plotter.py` Plots runtime/memory data (auto-detects by filename)

Task 1: Space Optimization in the 0-1 Knapsack Problem

Objective: Move from mathematical matrix logic to cache-friendly 1D array engineering to prevent Out-of-Memory (OOM) faults.

Variant	Strategy	Why It Matters
knapsack_2D	Full $(n + 1) \times (W + 1)$ table, one row per item	Simple to derive from the recurrence , but memory scales as $O(nW)$ and can exhaust RAM.
knapsack_1D	Single $W + 1$ array, updated in place per item	Collapses memory to $O(W)$, but only works correctly if the capacity loop runs <i>descending</i> .

0.1 TODO 1: Implement `knapsack_2D(n, W, weights, values)`

0.2 TODO 2: Implement `knapsack_1D(n, W, weights, values)`

Live Experimentation Setup

Runs to Perform

- **The Stress Test:** Set $n = 5,000$ and $W = 50,000$.
- **Memory Profiling:** Log the exact byte allocation of both models right before execution (e.g., `sizeof(int) * (n+1) * (W+1)` vs. `sizeof(int) * (W+1)`).

Compile & Run

```
1 g++ -O2 -o knapsack Knapsack.cpp
2 ./knapsack
3 python3 plotter.py data_knapsack.txt
```

Core Engineering Defense (Verbal Check)

- **The OOM Point:** Calculate the exact RAM footprint of the 2D matrix vs. the 1D array for large bounds. Explain why the 1D version bypasses the system memory limits that crash the 2D version.

The instructor will ask you to change the inner loop of the 1D array from descending (for w from W down to $\text{weights}[i]$) to ascending (for w from $\text{weights}[i]$ up to W). Run it, show the wrong output, and verbally explain the "**overwriting lookback**" bug — why an ascending loop silently turns the 0-1 Knapsack into an Unbounded Knapsack.